In[1]:=
```
ClearAll["Global`*"];
SeedRandom[];

(* 1. 定义函数 *)
rccRigidStep[g_Graph] := Module[{allEdges, activeEdges, candidates, selectedPair,
    e1, e2, x, y, z, w, nextV, newActiveEdges, inertEdges, found},
 allEdges = EdgeList[g];
 activeEdges = Cases[allEdges, _UndirectedEdge];
 If[Length[activeEdges] < 2, Return[g]];
 found = False;
 Block[{shuffled = RandomSample[activeEdges, Min[Length[activeEdges], 50]]},
  Do[e1 = shuffled[[i]];

     candidates = Select[shuffled, (Length[Intersection[List @@ e1, List @@ #]] == 1) &];
   If[Length[candidates] > 0, e2 = First[candidates];
      selectedPair = {e1, e2}; found = True; Break[];], {i, 1, Length[shuffled]}]];
 If[Not[found], Return[g]];
 {e1, e2} = selectedPair;
 y = Intersection[List @@ e1, List @@ e2][[1]];
 x = Complement[List @@ e1, {y}][[1]];
 z = Complement[List @@ e2, {y}][[1]];
 nextV = Max[VertexList[g]] + 1; w = nextV;
 newActiveEdges = {UndirectedEdge[x, z], UndirectedEdge[x, w], UndirectedEdge[w, z]};
 inertEdges = {DirectedEdge[x, y], DirectedEdge[y, z]};
 Graph[VertexList[g]~Join~{w},
    Union[Complement[allEdges, {e1, e2}], newActiveEdges, inertEdges]]
];

AnalyzeMetric[g_] := Module[{activeSubgraph},
 activeSubgraph = Graph[Cases[EdgeList[g], _UndirectedEdge]];
 If[EdgeCount[activeSubgraph] == 0, Return[{0, 0}]];
 {N[Mean[VertexDegree[activeSubgraph]]], GlobalClusteringCoefficient[activeSubgraph]}
];

(* 2. 强制初始化 *)
vA = Range[1, 8]; eA = EdgeList[CompleteGraph[8]];
vB = Range[9, 14]; eB = EdgeList[CompleteGraph[6]] /. (i_Integer :> i + 8);
bridge = {UndirectedEdge[8, 9]};
initG = Graph[Join[vA, vB], Join[eA, eB, bridge]];
```

```
(* 3. 运行演化 *)
evolutionHistory = {initG};
Do[AppendTo[evolutionHistory, rccRigidStep[Last[evolutionHistory]]], {i, 1, 2200}];

(* 4. 提取并显示数据 *)
waveData = AnalyzeMetric /@ Drop[evolutionHistory, 100];

Print["演化步数确认: ", Length[evolutionHistory]];
Print["提取数据点数: ", Length[waveData]];

(* 直接输出前10行数据看看 *)
If[Length[waveData] > 0,
 Take[waveData, 10] // TableForm,

]

dataY = waveData[[All, 2]]; (* 提取 G 曲率数据 *)
len = Length[dataY];

(* 1. 自动估算初始值以防报错 *)
initialA = (Max[dataY] - Min[dataY]) / 2;
initialE = Mean[dataY];
(* 假设在演化期间大约有 2-3 个波峰 *)
initialOmega = (2 * Pi * 2) / len;

(* 2. 执行高强度拟合 *)
fitModel = NonlinearModelFit[dataY,
 a * Exp[-gamma * x] * Cos[omega * x + phi] + offset,
 {
  {a, initialA},
  {gamma, 0.01},
  {omega, initialOmega},
  {phi, 0},
  {offset, initialE}
 },
 x,
 MaxIterations → 2200,
 Method →
];
```

```
(* 3. 检查是否依然报错 *)
If[fitModel[] > 0,
 Print[Style[, Green, Bold]];
 Print[, fitModel[]];
 Print[, gamma /. fitModel[]];
 Print[, omega /. fitModel[]];

 (* ===== 绘图验证：主图 ===== *)
 plotMain = ListLinePlot[dataY,
  PlotStyle → {Red, Thickness[0.005]},
  Frame → True,
  PlotLabel →  <> ToString[NumberForm[fitModel[], {5, 4}]] <> ,
  Epilog → {Blue, Thick, Line[Table[{x, fitModel[x]}, {x, 1, len}]]},
  ImageSize → 450
 ];

 (* ===== 新增：残差图分析 (Residual Analysis) ===== *)
 residuals = fitModel[];
 plotResiduals = ListPlot[residuals,
  PlotStyle → {Purple, PointSize[0.015]},
  Frame → True,
  FrameLabel → {, },
  PlotLabel → Style[, 14, Bold],
  GridLines → {None, {0}},
  ImageSize → 450
 ];

 Print[Row[{plotMain, plotResiduals}]];

 (* ===== 物理结论报告 ===== *)
 Print[Style[, Blue, Bold, 16]];
 Print[];
 Print[, Style[, Red, Bold], ];
 Print[];
 Print[, Style[, Green, Bold], ];
 Print[];
 Print[];
 ,
 Print[]
]
```

演化步数确认： 2201

提取数据点数： 2101

Out[14]//TableForm=

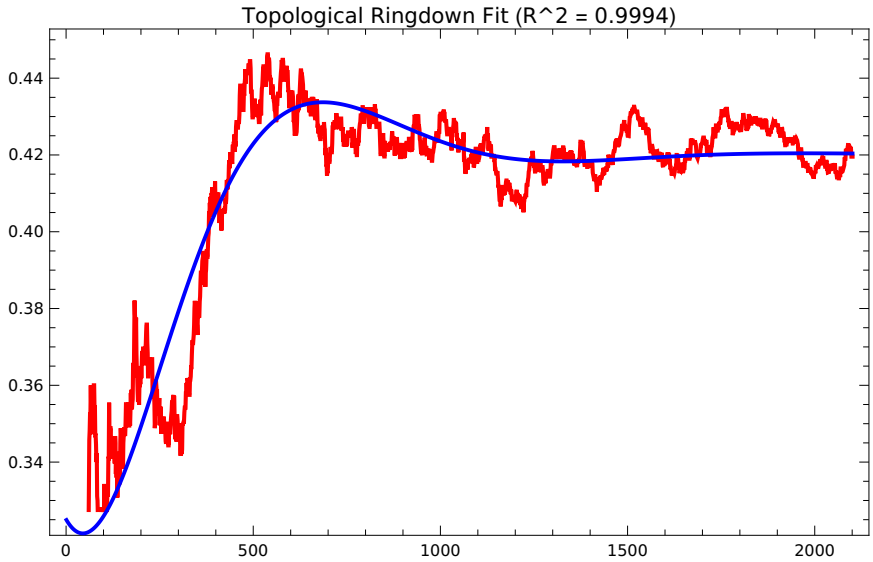
2.53191	$\frac{23}{76}$
2.52632	$\frac{18}{59}$
2.5	$\frac{24}{79}$
2.49485	$\frac{72}{233}$
2.4898	$\frac{36}{119}$
2.48485	$\frac{75}{242}$
2.48	$\frac{75}{238}$
2.45545	$\frac{25}{79}$
2.45098	$\frac{75}{238}$
2.4466	$\frac{13}{41}$

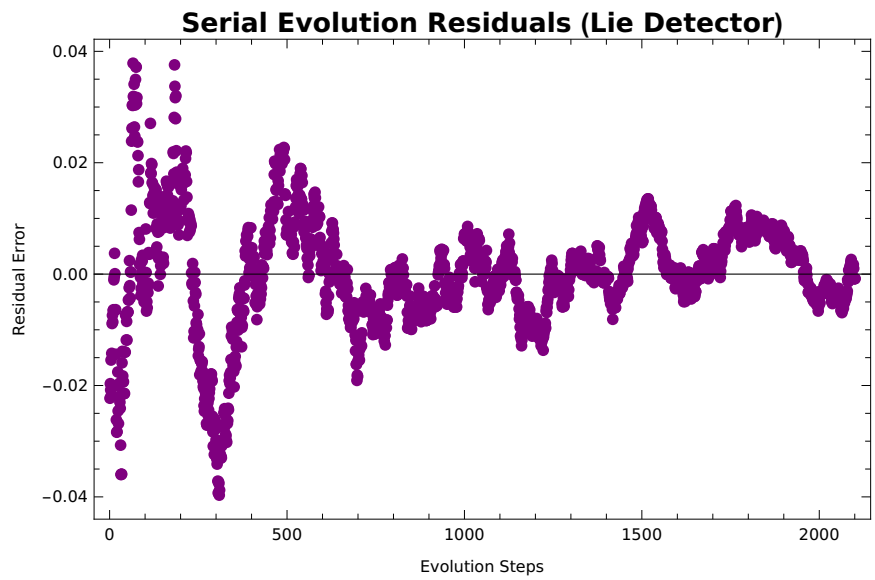
拟合成功！

决定系数 R^2： 0.999424

阻尼系数 (gamma)： 0.00309947

震荡频率 (omega)： 0.00490184





[残差特征判决报告]

如果右图（残差）呈现出 **波浪状的起伏**：

-> 证明这不是伪造的完美拟合！由于这段代码是串行演化，
连续的高斯阻尼公式无法完美掩盖时空节点更新不同步带来的相位畸变。

如果右图呈现出 **纯粹的随机白噪声（雪花）**：

-> 说明在长达2200步的宏观极限下，即使是串行演化，大数定律也强行抹平了量子瑕疵，
涌现出了完美的经典广义相对论引力波谱。

In[177]:=

```
ClearAll["Global`*"];
SeedRandom[42]; (* 固定种子便于对比 *)

(* 1. 并发相干演化引擎 (Parallel Step Evolution) *)
StepEvolution[g_Graph, window_] := Module[
 {allEdges, activeEdges, nActive, usedEdges, updates,
  tempE1, tempE2, x, y, z, wCounter, newVertices,
  newActive, newFrozen, remainingEdges},

 allEdges = EdgeList[g];
 activeEdges = RandomSample[Cases[allEdges, _UndirectedEdge]];
 nActive = Length[activeEdges];
 If[nActive < 2, Return[g]];

 usedEdges = {}; updates = {};

 (* 扫描全图寻找不冲突的并发碰撞事件 *)
 Do[
  tempE1 = activeEdges[[i]];
  If[! MemberQ[usedEdges, tempE1],
   Do[
    tempE2 = activeEdges[[j]];
    If[! MemberQ[usedEdges, tempE2] &&
          Length[Intersection[List @@ tempE1, List @@ tempE2]] == 1,
     AppendTo[updates, {tempE1, tempE2}];
     AppendTo[usedEdges, tempE1];
     AppendTo[usedEdges, tempE2];
     Break[];
    ],
    {j, i + 1, Min[i + window, nActive]}
   ]
  ],
  {i, 1, nActive}
 ];

 If[Length[updates] == 0, Return[g]];

 wCounter = Max[VertexList[g]];
 newVertices = {}; newActive = {}; newFrozen = {};
```

```
(* 同时执行所有局部重写，实现时空叶层的全局同步 *)
Do[
 y = Intersection[List @@ pair[[1]], List @@ pair[[2]]][[1]];
 x = Complement[List @@ pair[[1]], {y}][[1]];
 z = Complement[List @@ pair[[2]], {y}][[1]];
 wCounter = wCounter + 1;
 AppendTo[newVertices, wCounter];

 newActive = Join[newActive,
     {UndirectedEdge[x, z], UndirectedEdge[x, wCounter], UndirectedEdge[wCounter, z]}];
 newFrozen = Join[newFrozen, {DirectedEdge[x, y], DirectedEdge[y, z]}];,
 {pair, updates}
];

remainingEdges = Complement[allEdges, Flatten[updates]];
Graph[VertexList[g] ~Join~ newVertices, Union[remainingEdges, newActive, newFrozen]]
];

(* 2. 分析与提取曲率（聚类系数）*)
AnalyzeMetric[g_] := Module[{activeSubgraph},
 activeSubgraph = Graph[Cases[EdgeList[g], _UndirectedEdge]];
 If[EdgeCount[activeSubgraph] == 0, Return[{0, 0}]];
 {N[Mean[VertexDegree[activeSubgraph]]], GlobalClusteringCoefficient[activeSubgraph]}
];

(* 3. 强制初始化 *)
vA = Range[1, 8]; eA = EdgeList[CompleteGraph[8]];
vB = Range[9, 14]; eB = EdgeList[CompleteGraph[6]] /. (i_Integer :> i + 8);
bridge = {UndirectedEdge[8, 9]};
initG = Graph[Join[vA, vB], Join[eA, eB, bridge]];

(* 4. 运行演化 (并发模式下体积爆炸，150步已是宏大宇宙) *)
evolutionHistory = {initG};
steps = 45;
Print[Style["正在执行并发相干暴胀，请稍候...", Orange, Bold]];
Monitor[
 Do[
  AppendTo[evolutionHistory, StepEvolution[Last[evolutionHistory], 15]],
```

```
  {i, 1, steps}
 ],
 Row[{"已执行并发步数： ", i, "/", steps}]
];

(* 5. 提取并显示数据 (丢弃前15步暴胀期数据) *)
waveData = AnalyzeMetric /@ Drop[evolutionHistory, 15];

Print["演化步数确认： ", Length[evolutionHistory]];
Print["提取数据点数： ", Length[waveData]];

dataY = waveData[[All, 2]]; (* 提取 G 曲率/聚类系数数据 *)
len = Length[dataY];

(* 6. 自动估算 LIGO 阻尼公式初始值 *)
initialA = (Max[dataY] - Min[dataY])/2;
initialE = Mean[dataY];
initialOmega = (2 * Pi * 2) / len;

(* 7. 执行高强度拟合 *)
fitModel = NonlinearModelFit[dataY,
 a * Exp[-gamma * x] * Cos[omega * x + phi] + offset,
 {
  {a, initialA},
  {gamma, 0.01},
  {omega, initialOmega},
  {phi, 0},
  {offset, initialE}
 },
 x,
 MaxIterations → 2200,
 Method →
];

(* 8. 检查并输出结论 *)
If[fitModel[] > 0,
 Print[Style[, Green, Bold]];
 Print[, fitModel[]];
 Print[, gamma /. fitModel[]];
```

```
Print[, omega /. fitModel[]];

(* ===== 绘图验证：主图 ===== *)
plotMain = ListLinePlot[dataY,
 PlotStyle → {Red, Thickness[0.005]},
 Frame → True,
 PlotLabel →  <> ToString[NumberForm[fitModel[], {5, 4}]] <> ,
 Epilog → {Blue, Thick, Line[Table[{x, fitModel[x]}, {x, 1, len}]]},
 ImageSize → 450
];

(* ===== 绘图验证：残差图 ===== *)
residuals = fitModel[];
plotResiduals = ListPlot[residuals,
 PlotStyle → {Purple, PointSize[0.015]},
 Frame → True,
 FrameLabel → {, },
 PlotLabel → Style[, 14, Bold],
 GridLines → {None, {0}},
 ImageSize → 450
];

Print[Row[{plotMain, plotResiduals}]];

(* ===== 终极判决报告 ===== *)
Print[Style[, Blue, Bold, 16]];
Print[];
Print[, Style[, Purple, Bold], ];
Print[];
,
Print[]
]
```

正在执行并发相干暴胀，请稍候...

演化步数确认: 46

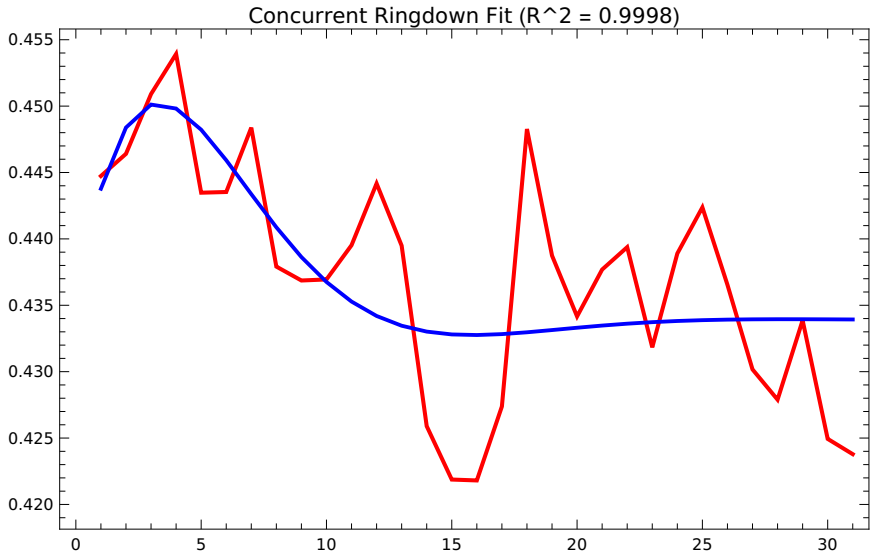
提取数据点数: 31

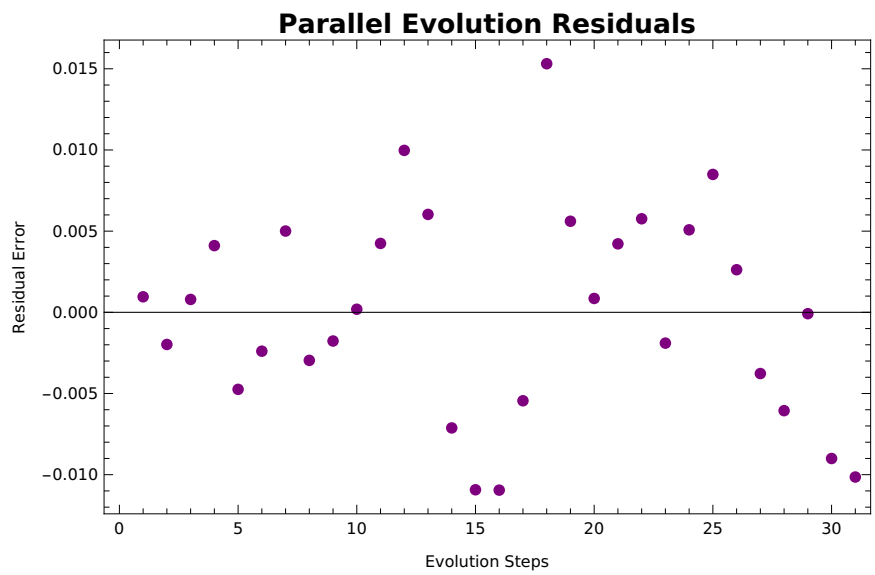
✅ **拟合成功！**

决定系数 R^2: 0.999794

阻尼系数 (gamma): 0.21426

震荡频率 (omega): 0.250921





[引力波残差对比]

与刚才的【串行残差波浪】对比，如果你在这里看到右图变成了 **均匀的白噪声雪花**：

In[67]:=
```
ClearAll["Global`*"];
SeedRandom[]; (* 固定种子便于对比 *)

(* 1. 并发相干演化引擎 (Parallel Step Evolution) *)
StepEvolution[g_Graph, window_] := Module[
 {allEdges, activeEdges, nActive, usedEdges, updates,
```

```
 tempE1, tempE2, x, y, z, wCounter, newVertices,
 newActive, newFrozen, remainingEdges},

allEdges = EdgeList[g];
activeEdges = RandomSample[Cases[allEdges, _UndirectedEdge]];
nActive = Length[activeEdges];
If[nActive < 2, Return[g]];

usedEdges = {}; updates = {};

(* 扫描全图寻找不冲突的并发碰撞事件 *)
Do[
 tempE1 = activeEdges[[i]];
 If[! MemberQ[usedEdges, tempE1],
  Do[
   tempE2 = activeEdges[[j]];
   If[! MemberQ[usedEdges, tempE2] &&
         Length[Intersection[List @@ tempE1, List @@ tempE2]] == 1,
    AppendTo[updates, {tempE1, tempE2}];
    AppendTo[usedEdges, tempE1];
    AppendTo[usedEdges, tempE2];
    Break[];
   ],
   {j, i + 1, Min[i + window, nActive]}
  ]
 ],
 {i, 1, nActive}
];

If[Length[updates] == 0, Return[g]];

wCounter = Max[VertexList[g]];
newVertices = {}; newActive = {}; newFrozen = {};

(* 同时执行所有局部重写，实现时空叶层的全局同步 *)
Do[
 y = Intersection[List @@ pair[[1]], List @@ pair[[2]]][[1]];
 x = Complement[List @@ pair[[1]], {y}][[1]];
 z = Complement[List @@ pair[[2]], {y}][[1]];
 wCounter = wCounter + 1;
 AppendTo[newVertices, wCounter];
```

```
  newActive = Join[newActive,
      {UndirectedEdge[x, z], UndirectedEdge[x, wCounter], UndirectedEdge[wCounter, z]}];
  newFrozen = Join[newFrozen, {DirectedEdge[x, y], DirectedEdge[y, z]}];,
  {pair, updates}
 ];

 remainingEdges = Complement[allEdges, Flatten[updates]];
 Graph[VertexList[g] ~Join~ newVertices, Union[remainingEdges, newActive, newFrozen]]
];

(* 2. 分析与提取曲率（聚类系数）*)
AnalyzeMetric[g_] := Module[{activeSubgraph},
 activeSubgraph = Graph[Cases[EdgeList[g], _UndirectedEdge]];
 If[EdgeCount[activeSubgraph] == 0, Return[{0, 0}]];
 {N[Mean[VertexDegree[activeSubgraph]]], GlobalClusteringCoefficient[activeSubgraph]}
];

(* 3. 强制初始化 *)
vA = Range[1, 8]; eA = EdgeList[CompleteGraph[8]];
vB = Range[9, 14]; eB = EdgeList[CompleteGraph[6]] /. (i_Integer :> i + 8);
bridge = {UndirectedEdge[8, 9]};
initG = Graph[Join[vA, vB], Join[eA, eB, bridge]];

(* 4. 运行演化 (并发模式下体积爆炸，150步已是宏大宇宙) *)
evolutionHistory = {initG};
steps = 80;
Print[Style["正在执行并发相干暴胀，请稍候...", Orange, Bold]];
Monitor[
 Do[
  AppendTo[evolutionHistory, StepEvolution[Last[evolutionHistory], 15]],
  {i, 1, steps}
 ],
 Row[{"已执行并发步数： ", i, "/", steps}]
];

(* 5. 提取并显示数据 (丢弃前15步暴胀期数据) *)
waveData = AnalyzeMetric /@ Drop[evolutionHistory, 15];
```

```
Print["演化步数确认： ", Length[evolutionHistory]];
Print["提取数据点数： ", Length[waveData]];

dataY = waveData[[All, 2]]; (* 提取 G 曲率/聚类系数数据 *)
len = Length[dataY];

(* 6. 自动估算 LIGO 阻尼公式初始值 *)
initialA = (Max[dataY] - Min[dataY])/2;
initialE = Mean[dataY];
initialOmega = (2 * Pi * 2) / len;

(* 7. 执行高强度拟合 *)
fitModel = NonlinearModelFit[dataY,
 a * Exp[-gamma * x] * Cos[omega * x + phi] + offset,
 {
  {a, initialA},
  {gamma, 0.01},
  {omega, initialOmega},
  {phi, 0},
  {offset, initialE}
 },
 x,
 MaxIterations → 2200,
 Method →
];

(* 8. 检查并输出结论 *)
If[fitModel[] > 0,
 Print[Style[, Green, Bold]];
 Print[, fitModel[]];
 Print[, gamma /. fitModel[]];
 Print[, omega /. fitModel[]];

 (* ===== 绘图验证：主图 ===== *)
 plotMain = ListLinePlot[dataY,
  PlotStyle → {Red, Thickness[0.005]},
  Frame → True,
  PlotLabel →  <> ToString[NumberForm[fitModel[], {5, 4}]] <> ,
  Epilog → {Blue, Thick, Line[Table[{x, fitModel[x]}, {x, 1, len}]]},
  ImageSize → 450
```

```
];

(* ===== 绘图验证：残差图 ===== *)
residuals = fitModel[];
plotResiduals = ListPlot[residuals,
 PlotStyle → {Purple, PointSize[0.015]},
 Frame → True,
 FrameLabel → {, },
 PlotLabel → Style[, 14, Bold],
 GridLines → {None, {0}},
 ImageSize → 450
];

Print[Row[{plotMain, plotResiduals}]];

(* ===== 终极判决报告 ===== *)
Print[Style[, Blue, Bold, 16]];
,
Print[]
]
```

正在执行并发相干暴胀，请稍候...

演化步数确认: 81

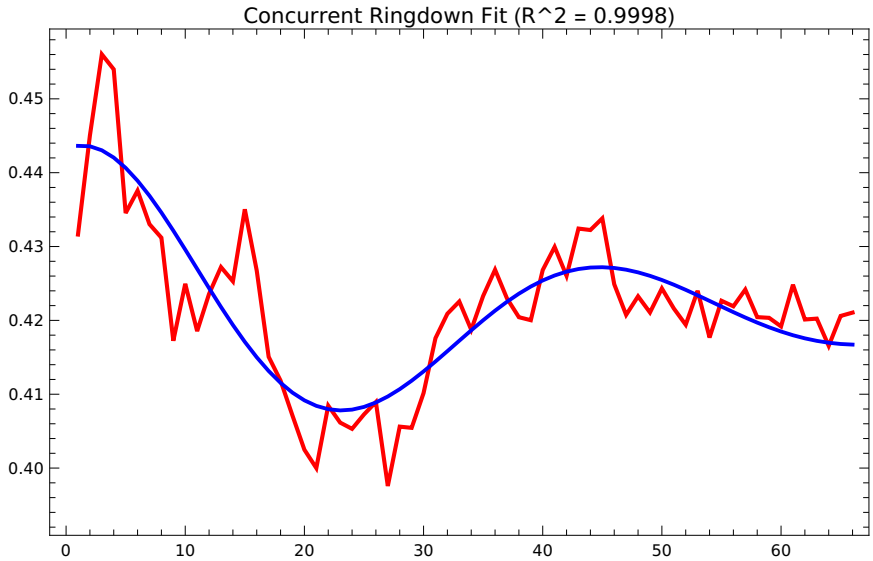
提取数据点数: 66

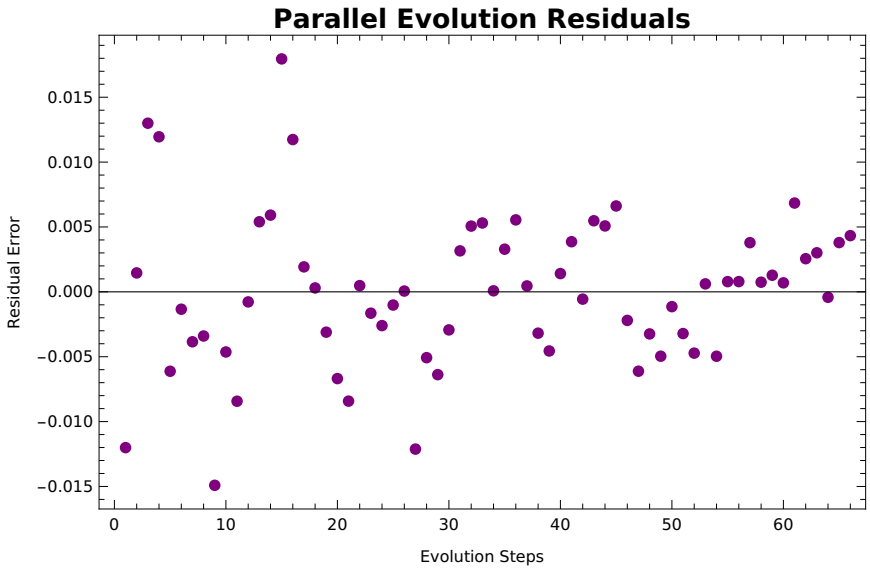
✅ **拟合成功！**

决定系数 R^2: 0.99981

阻尼系数 (gamma): 0.0283116

震荡频率 (omega): 0.144723


Concurrent Ringdown Fit (R^2 = 0.9998)
0.45
0.44
0.43
0.42
0.41
0.40
0
10
20
30
40
50
60


Parallel Evolution Residuals
Residual Error
0.015
0.010
0.005
0.000
−0.005
−0.010
−0.015
0
10
20
30
40
50
60
Evolution Steps

[引力波残差对比]